

FACULTY OF COMPUTING & INFORMATICS (FCI)

CSN6214 OPERATING SYSTEMS ASSIGNMENT TT2L

Trimester October/November 2025, Term 2530

Student ID	Name
242UC244R9	Beatrice Lew Yu Ling
242UC244FH	Ee Yu Suen
242UC244PD	Tan Yi Ling

1. Game Description & Rules

1.1 Game Description

This is an multi-player expansion of the traditional Tic-Tac-Toe game. The system offers scalable gameplay with multiple board configurations, including 3x3, 4x4 or 6x6 grid. Players take turns selecting a cell by entering its corresponding grid number. The objective is to form a continuous line horizontally, vertically, or diagonally to achieve victory.

1.2 Rules

Player Registration: The system manages player entry using a First-In, First-Out (FIFO) method. The first person connected to the game is designated as Player 1, who acts as the Host with the authority to decide the total number of players for the game. Subsequent users who connect are designated as Player 2, Player 3, and so on, based on their arrival time.

Lobby Phase: Once the host sets the player count, the game enters a lobby state where all players must wait until the required number of players have joined the game. Once the maximum number of players has been reached, the system will reject any further connection attempts.

Symbol Selection: Starting with Player 1, each player chooses a unique symbol (A-Z). The uniqueness of symbols is strictly enforced to prevent visual confusion and ensure accurate move tracking.

Gameplay Execution: During the game, players take turns entering a grid number corresponding to the cell where they wish to place their symbol. If a player does not respond within a few seconds, their turn is automatically skipped and passed to the next player.

Winning Conditions: A player wins by successfully forming a horizontal, vertical, or diagonal line of their symbols and the respective player is awarded one point.

Draw Conditions: If the grid is completely filled without any player achieving a line, the game results in a Draw.

2. Deployment Mode

The system is deployed in Single Machine Mode, where the server and all clients are located on the same physical hardware, using local Inter-Process Communication (IPC) mechanisms for high-speed data exchange.

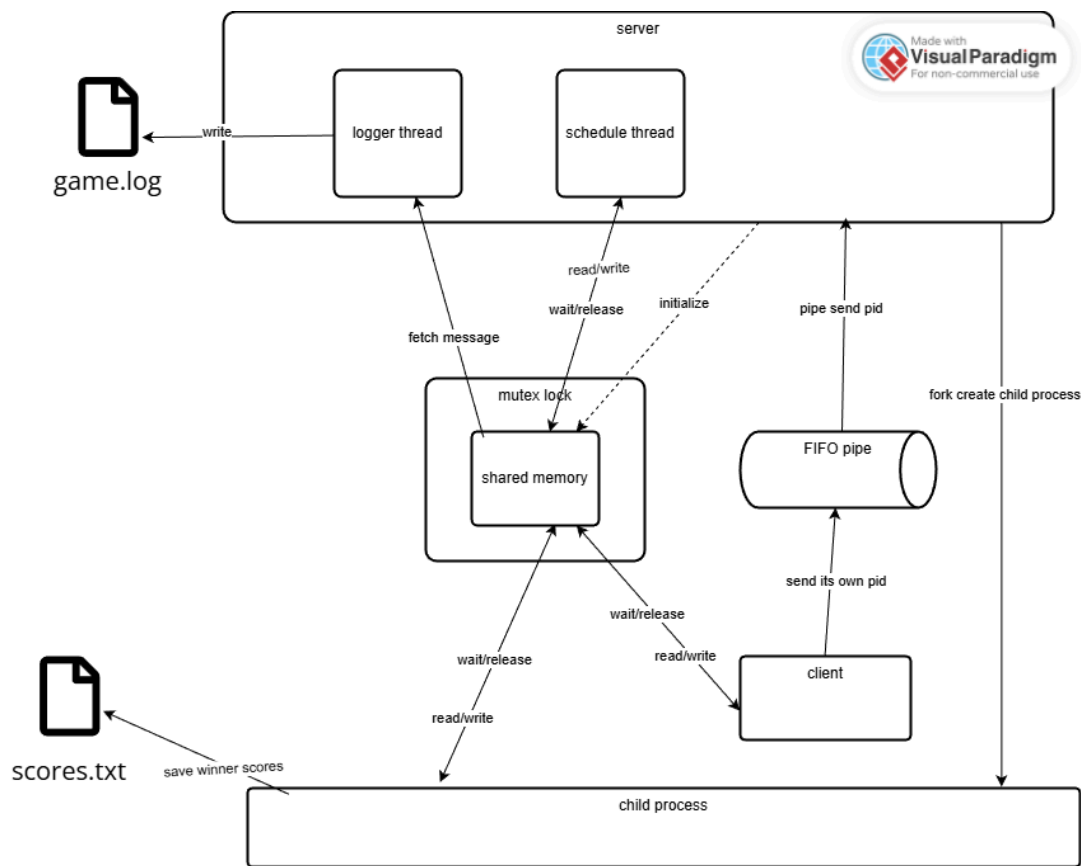
Server: A single primary process responsible for environment setup. It initializes the "game room" by creating the Shared Memory segment and the Named Pipe (FIFO) before any clients connect.

Internal Threads: Within the primary server process, specialized threads are spawned to handle background administrative tasks. A Scheduler Thread manages turn timing and player skipping, while a Logger Thread handles writing game events to disk, ensuring these critical tasks run concurrently without blocking the main server operations.

Child Processes: Every time a new player joins, the server does not handle the logic directly. Instead, it "clones" itself using the **fork()** system call to create a dedicated helper process for that specific player.

Local Clients: Multiple client instances (3 to 5 players) are executed on the same machine.

3. Hybrid Architecture



The diagram illustrates the data and control flow within the Hybrid Architecture.

1. Connection phase

The process begins when the Client sends its own PID through the FIFO Pipe. This is the only time a pipe is used, acting as a "gatekeeper" allowing the Parent Process to acknowledge and verify new connections before spawning a dedicated handler.

2. Shared memory

Data storage: It is utilized to share game data among clients, including the board state, player symbols, and real-time scores.

Concurrency control: Access to the shared memory is governed by Process-Shared Mutexes. These prevent data corruption and race conditions when multiple processes attempt to access the memory simultaneously.

3. Functional Roles and Interactions

Child process: Each child process performs bidirectional communication with Shared Memory. It reads the user's intent moves, validates it against game rules, and writes back the updated board and winner status.

Logger thread: This thread follows a unidirectional data flow from Shared Memory. It fetches messages from the circular buffer and writes them to game.log. It uses the log_mutex to safely interact with the buffer.

Schedule thread: This thread monitors the player's turn duration in Shared Memory. If it detects a timeout 20 seconds, it forcibly modifies the player turn index to maintain game flow.

4. IPC mechanism & Shared Memory Layout

Named Pipe (FIFO): Acts as the "gatekeeper." allowing the Client to send its PID to the Parent Process to request entry into the game.

Shared Memory: Acts as the "Game Table." Once the game starts, all communication (moves, board updates, and scores) happens here.

Mutex Synchronization: To prevent data corruption, a Process-Shared Mutex is used to ensure that only one process can write to the Shared Memory at any given time.

4.1 Shared Memory Layout

Variable name	Description
board[BOARD_SIZE];	Represents the game grid
symbols[MAX_PLAYERS];	Stores the unique character (A-Z) chosen by each player.
player_pids[MAX_PLAYERS];	Stores the Process IDs of connected clients to track active sessions.
player_scores[MAX_PLAYERS]	Persists the score of each player throughout the game session.
pending_moves[MAX_PLAYERS];	Acts as a "mailbox" where the Client writes their move for the Child Process to process.

last_move_result[MAX_PLAYERS]	Used by the Child Process to send feedback to the Client
current_player_index	Tracks which player's turn it is.
game_phase	Manages the transitions between Lobby, Symbol Selection, and Gameplay phases.
game_over & winner	Flags to signal the end of the game and identify the victorious player.
time_t turn_start_time	Used by the schedule thread to enforce the 20-second turn limit.
char global_message[256]	A character buffer for broadcasting system announcements to all Clients.
game_mutex & log_mutex	Process-shared mutexes
log_queue[MAX_LOGS][LOG_LEN]	A dedicated area for the Producer-Consumer logging system, managed by log_head, log_tail, and log_count.

5. Synchronization Strategy

5.1 Process-Shared Mutexes

Standard mutexes only work between threads. Since our system uses `fork()`, we initialized our mutexes with the Process-Shared Attribute (`PTHREAD_PROCESS_SHARED`). This allows the Parent Process, Child Processes, and Threads to respect the same locks residing in Shared Memory.

5.2 Critical Sections & Coordination

We defined two distinct critical sections:

The Game Lock (`game_mutex`): Protects the board and turn variables. It ensures that two players cannot modify the grid at the exact same microsecond.

The Log Lock (`log_mutex`): Protects the `log_queue` indices (head and tail). This allows the Logger Thread to read while Game Processes write without corrupting the buffer.

6. Internal Thread Subsystem

6.1 Concurrent Logger Design

To prevent slow disk I/O from blocking the real-time game logic, we implemented a Non-Blocking Logger using the Producer-Consumer pattern.

The Buffer: We utilize a circular buffer (`log_queue`) residing in Shared Memory.

The Producers: Game processes push strings to the buffer and update the tail index.

The Consumer: A dedicated `logger_thread` wakes up, reads from the head index, and writes to `game.log`. This decouples file writing from gameplay.

6.2 Round Robin Scheduler Logic

A dedicated `schedule_thread` runs continuously to enforce fairness:

It checks `difftime(now, turn_start_time)` every second.

If the time exceeds 20 seconds, it acquires the `game_mutex`.

It forcibly skips the turn: `current_player = (current + 1) % players`.

It logs a "Forced Skip" event, ensuring the game never stalls due to an inactive player.

7. Persistence & Data Management

7.1 File Format (`scores.txt`)

We utilize a simple text-based flat file format. Each line corresponds to a player slot's total wins (Player 1 to Player 5).

7.2 Loading & Saving Mechanism

Loading: On startup, the server reads `scores.txt` into the `player_scores[]` array in shared memory.

Saving: Saving is triggered by a Signal Handler. When the server receives a SIGINT (Ctrl+C), it blocks all threads, writes the current `player_scores[]` from memory back to `scores.txt`, and then unlinks shared memory resources. This guarantees no data is lost upon shutdown.

8. Multi-Game Handling

8.1 Game Reset Mechanism

The system supports continuous play without restarting the server process. Upon receiving an INIT command from a new host:

Memory Wipe: The `init_game()` function clears the board and resets log indices.

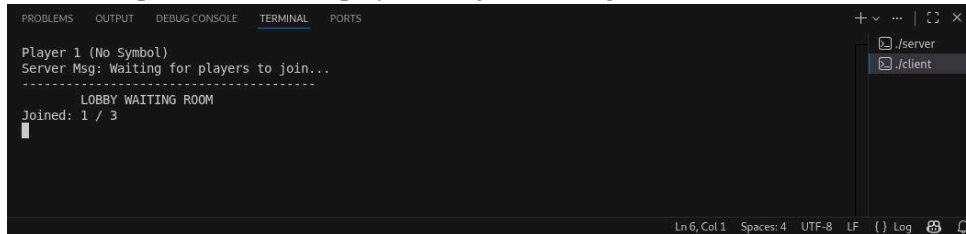
Thread Continuity: The Scheduler and Logger threads (which never exit) simply see the reset variables and continue processing for the new session.

Zombie Cleanup: A SIGCHLD handler cleans up the processes from the previous game to prevent resource leaks.

9. Testing & Verification

9.1 Gameplay Evidence

shows the lobby phase, where the host waits until the required number of players has joined the game.



```
Player 1 (No Symbol)
Server Msg: Waiting for players to join...
-----
LOBBY WAITING ROOM
Joined: 1 / 3
```

shows the symbol selection phase, where non-host clients are required to wait until the host selects a symbol first.

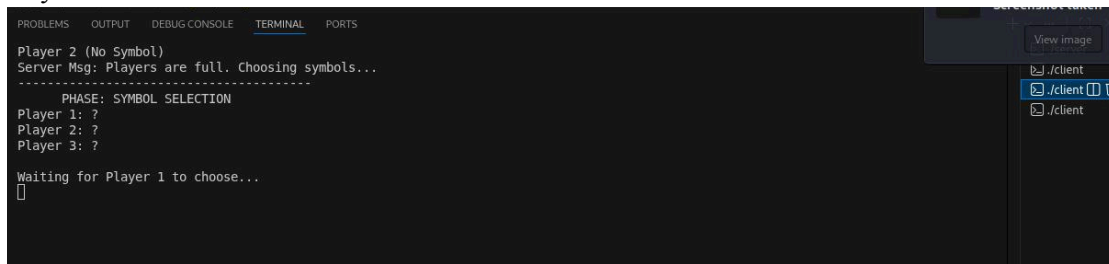
Player 1(host):



```
Player 1 (No Symbol)
Server Msg: Players are full. Choosing symbols...
-----
PHASE: SYMBOL SELECTION
Player 1: ?
Player 2: ?
Player 3: ?

>>> YOUR TURN TO CHOOSE SYMBOL! <<<
Enter a letter (A-Z):
```

Player 2:

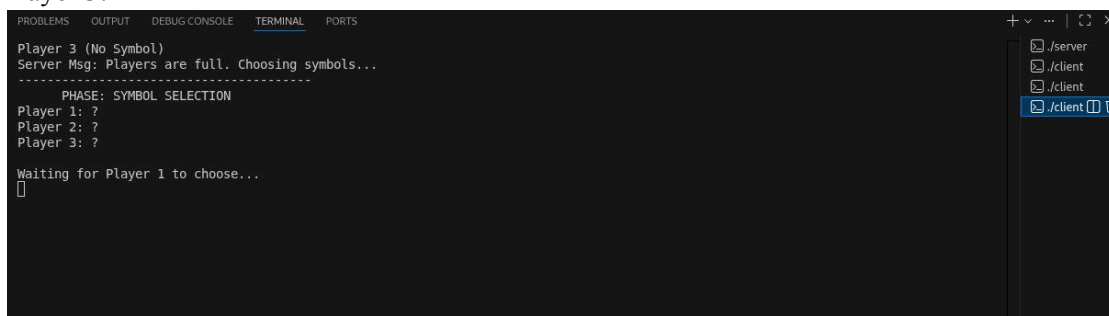


```
Player 2 (No Symbol)
Server Msg: Players are full. Choosing symbols...
-----
PHASE: SYMBOL SELECTION
Player 1: ?
Player 2: ?
Player 3: ?

Waiting for Player 1 to choose...

```

Player 3:



```
Player 3 (No Symbol)
Server Msg: Players are full. Choosing symbols...
-----
PHASE: SYMBOL SELECTION
Player 1: ?
Player 2: ?
Player 3: ?

Waiting for Player 1 to choose...

```

show multiple client terminals displaying the same game completion result after the server detects a winning condition.

Player 1:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Player 1 (A)
Server Msg: Player 1 Wins!
-----
=====GRID LABELS=====
 1 | 2 | 3 | 4
--|--|--|--
 5 | 6 | 7 | 8
--|--|--|--
 9 |10 |11 |12
--|--|--|--
13 |14 |15 |16
--|--|--|--

===== GAME BOARD =====
Players: P1:[A] P2:[B] P3:[C]
 A | B | C | A
--|--|--|--
 B | C | A | B
--|--|--|--
 C | A | B | C
--|--|--|--
 A |   |   |
--|--|--|--

GAME OVER!
YOU WIN!!!
Ln 26, Col 1 Spaces: 4
```

Player 2:

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
Player 2 (B)
Server Msg: Player 1 Wins!
-----
=====GRID LABELS=====
 1 | 2 | 3 | 4
--|--|--|--
 5 | 6 | 7 | 8
--|--|--|--
 9 |10 |11 |12
--|--|--|--
13 |14 |15 |16
--|--|--|--

===== GAME BOARD =====
Players: P1:[A] P2:[B] P3:[C]
 A | B | C | A
--|--|--|--
 B | C | A | B
--|--|--|--
 C | A | B | C
--|--|--|--
 A |   |   |
--|--|--|--

GAME OVER!
Player 1 wins.
Ln 2
```

Player 3:

File Edit Shell View Window Terminal Help

Player 3 (C)
Server Msg: Player 1 Wins!

====GRID LABELS====

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

===== GAME BOARD =====
Players: P1:[A] P2:[B] P3:[C]

A	B	C	A
B	C	A	B
C	A	B	C
A			

GAME OVER!
Player 1 wins.

shows automatic turn skipping after timeout,advancing the turn to the next player.

Player 2 (B)
Server Msg: Time Up! Skipped Player 2.

====GRID LABELS====

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

===== GAME BOARD =====
Players: P1:[A] P2:[B] P3:[C]

A	B	C	A

Waiting for Player 3 (C)...

|

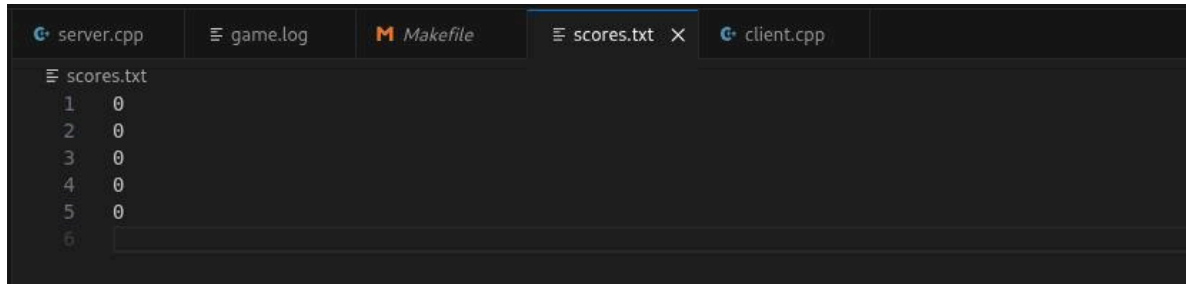
9.2 Concurrency Evidence (game.log)

```
[2026-02-07 13:34:58] Game Initialized.
[2026-02-07 13:34:58] Persistence: Scores loaded.
[2026-02-07 13:34:58] System: Logger Thread Started.
[2026-02-07 13:34:58] Scheduler Started.
[2026-02-07 13:35:04] Game Initialized.
[2026-02-07 13:35:09] [Game] Phase changed to SYMBOL_SELECT.
[2026-02-07 13:35:12] Player 1 selected symbol 'A'
[2026-02-07 13:35:13] Player 2 selected symbol 'B'
[2026-02-07 13:35:15] Player 3 selected symbol 'C'
[2026-02-07 13:35:15] [Game] Phase changed to GAMEPLAY.
[2026-02-07 13:35:17] Player 1 (A) placed symbol on Grid 1
[2026-02-07 13:35:17] Turn Change: It is now Player 2's turn
[2026-02-07 13:35:18] Player 2 (B) placed symbol on Grid 2
[2026-02-07 13:35:18] Turn Change: It is now Player 3's turn
[2026-02-07 13:35:20] Player 3 (C) placed symbol on Grid 3
[2026-02-07 13:35:20] Turn Change: It is now Player 1's turn
[2026-02-07 13:35:23] Player 1 (A) placed symbol on Grid 4
[2026-02-07 13:35:23] Turn Change: It is now Player 2's turn
[2026-02-07 13:35:24] Player 2 (B) placed symbol on Grid 5
[2026-02-07 13:35:24] Turn Change: It is now Player 3's turn
[2026-02-07 13:35:26] Player 3 (C) placed symbol on Grid 6
[2026-02-07 13:35:26] Turn Change: It is now Player 1's turn
[2026-02-07 13:35:29] Player 1 (A) placed symbol on Grid 7
[2026-02-07 13:35:29] Turn Change: It is now Player 2's turn
[2026-02-07 13:35:31] Player 2 (B) placed symbol on Grid 8
[2026-02-07 13:35:31] Turn Change: It is now Player 3's turn
[2026-02-07 13:35:34] Player 3 (C) placed symbol on Grid 9
[2026-02-07 13:35:34] Turn Change: It is now Player 1's turn
[2026-02-07 13:35:37] Player 1 (A) placed symbol on Grid 10
[2026-02-07 13:35:37] Turn Change: It is now Player 2's turn
[2026-02-07 13:35:39] Player 2 (B) placed symbol on Grid 11
[2026-02-07 13:35:39] Turn Change: It is now Player 3's turn
[2026-02-07 13:35:42] Player 3 (C) placed symbol on Grid 12 |
[2026-02-07 13:35:42] Turn Change: It is now Player 1's turn
[2026-02-07 13:35:44] Player 1 (A) placed symbol on Grid 13
[2026-02-07 13:35:44] Persistence: Scores saved.
[2026-02-07 13:35:44] RESULT: GAME OVER. The Winner is Player 1!
[2026-02-07 13:38:32] Game Initialized.
```

```
[2026-02-07 13:38:32] Game Initialized.
[2026-02-07 13:38:32] Persistence: Scores loaded.
[2026-02-07 13:38:32] System: Logger Thread Started.
[2026-02-07 13:38:32] Scheduler Started.
[2026-02-07 13:38:37] Game Initialized.
[2026-02-07 13:38:42] [Game] Phase changed to SYMBOL_SELECT.
[2026-02-07 13:38:44] Player 1 selected symbol 'A'
[2026-02-07 13:38:46] Player 2 selected symbol 'B'
[2026-02-07 13:38:48] Player 3 selected symbol 'C'
[2026-02-07 13:38:48] [Game] Phase changed to GAMEPLAY.
[2026-02-07 13:38:50] Player 1 (A) placed symbol on Grid 1
[2026-02-07 13:38:50] Turn Change: It is now Player 2's turn
[2026-02-07 13:38:52] Player 2 (B) placed symbol on Grid 2
[2026-02-07 13:38:52] Turn Change: It is now Player 3's turn
[2026-02-07 13:38:54] Player 3 (C) placed symbol on Grid 3
[2026-02-07 13:38:54] Turn Change: It is now Player 1's turn
[2026-02-07 13:38:57] Player 1 (A) placed symbol on Grid 4
[2026-02-07 13:38:57] Turn Change: It is now Player 2's turn
[2026-02-07 13:38:59] Player 2 (B) placed symbol on Grid 5
[2026-02-07 13:38:59] Turn Change: It is now Player 3's turn
[2026-02-07 13:39:02] Player 3 (C) placed symbol on Grid 6
[2026-02-07 13:39:02] Turn Change: It is now Player 1's turn
[2026-02-07 13:39:05] Player 1 (A) placed symbol on Grid 7
[2026-02-07 13:39:05] Turn Change: It is now Player 2's turn
[2026-02-07 13:39:08] Player 2 (B) placed symbol on Grid 8
[2026-02-07 13:39:08] Turn Change: It is now Player 3's turn
[2026-02-07 13:39:10] Player 3 (C) placed symbol on Grid 9
[2026-02-07 13:39:10] Turn Change: It is now Player 1's turn
[2026-02-07 13:39:13] Player 1 (A) placed symbol on Grid 10
[2026-02-07 13:39:13] Turn Change: It is now Player 2's turn
[2026-02-07 13:39:16] Player 2 (B) placed symbol on Grid 11
[2026-02-07 13:39:16] Turn Change: It is now Player 3's turn
[2026-02-07 13:39:18] Player 3 (C) placed symbol on Grid 12
[2026-02-07 13:39:18] Turn Change: It is now Player 1's turn
[2026-02-07 13:39:21] Player 1 (A) placed symbol on Grid 13
[2026-02-07 13:39:21] Persistence: Scores saved.
[2026-02-07 13:39:21] RESULT: GAME OVER. The Winner is Player 1!
[2026-02-07 13:39:31] Game Initialized.
```

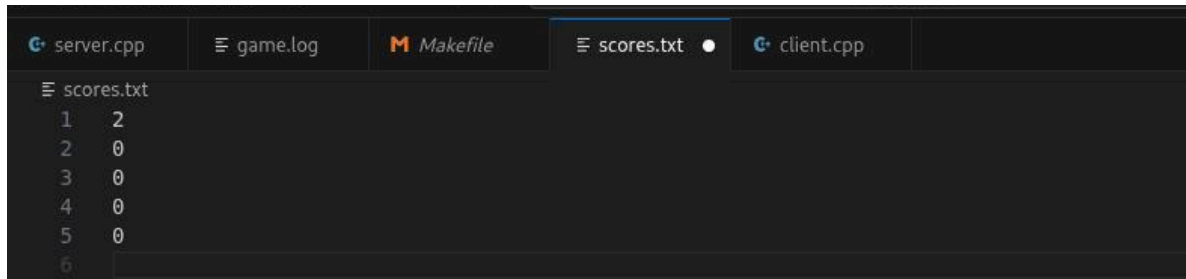
9.3 Persistence Evidence

Before:



```
server.cpp  game.log  Makefile  scores.txt  client.cpp
scores.txt
1 0
2 0
3 0
4 0
5 0
6
```

After:



```
server.cpp  game.log  Makefile  scores.txt  client.cpp
scores.txt
1 2
2 0
3 0
4 0
5 0
6
```

10. Team Responsibilities

Name/ID	Primary Role	Key Components Devel-oped/Implemented
Ee Yu Suen / 242UC244FH	Server Core, IPC	fork(), shared memory setup, and process-shared mutexes
Beatrice Lew Yu Ling / 242UC244R9	Internal threads and scheduling	Round Robin turn scheduler thread, concurrent logger threas, thread-safe queues, multi-game handling and server persistence
Tan Yi Ling	Game logic and data persistence	Client-side logic,server-enforced game rules,scores.txt loading/saving,and atomic score updates